

# aula12

April 24, 2026

## 1 Linguagens e Ambientes de Programação

### 1.1 (Aula Teórica 11)

#### 1.1.1 Agenda

- Árvores n-árias

#### 1.2 Árvores n-árias

Numa árvore n-ária (**ntree**) cada nó pode ter **qualquer número de filhos**, representados como uma lista de sub-árvores. Considere o tipo indutivo definido abaixo e o exemplo **nt** — uma árvore com raiz 1, subárvores com raízes 2, 4 e 5, e o sendo que a subarvore com o valor 2 na raiz tem uma subarvore com o valor 3.

```
[6]: type 'a ntree = NEmpty | NNode of 'a * 'a ntree list  
  
let nt = NNode (1, [NNode (2, [NNode (3, [])]); NNode (4, []); NNode (5, [])])
```

```
[6]: type 'a ntree = NEmpty | NNode of 'a * 'a ntree list
```

```
[6]: val nt : int ntree =  
      NNode (1, [NNode (2, [NNode (3, [])]); NNode (4, []); NNode (5, [])])
```

Esta outra definição de tipos é mais precisa no sentido em que não há confusão entre usar uma árvore vazia nas subárvores ou uma lista vazia. Vai requerer sempre que todas as funções tenham os casos para tratar a árvore vazia no nível de topo.

```
[ ]: type 'a netree = NNode of 'a * 'a netree list  
  
type 'a nntree = NEmpty | NNEmpty of 'a netree
```

```
[ ]: type 'a netree = NNode of 'a * 'a netree list
```

```
[ ]: type 'a nntree = NEmpty | NNEmpty of 'a netree
```

### 1.2.1 Soma de todos os valores numa árvore n-ária — versão 1 (com função auxiliar explícita)

`sum_tree` soma todos os valores da árvore. A função auxiliar `sum_aux` itera sobre a lista de filhos, chamando `sum_tree` em cada sub-árvore.

```
[9]: let rec sum_tree t =  
      let rec sum_aux l =  
        match l with  
        | [] -> 0  
        | t'::ts -> sum_tree t' + sum_aux ts  
      in  
      match t with  
      | NEmpty -> 0  
      | NNode(v, l) -> v + sum_aux l  
  
      let _ = assert (sum_tree nt = 15)
```

```
[9]: val sum_tree : int ntree -> int = <fun>
```

```
[9]: - : unit = ()
```

### 1.2.2 Soma dos valores — versão 2 (com `fold_left`)

Versão mais compacta que substitui a auxiliar explícita por `List.fold_left`. Nota: o acumulador inicial é `v` (o valor do nó atual), o que pode ser confuso — compare com a versão 4.

```
[ ]: let rec sum_tree t =  
      match t with  
      | NEmpty -> 0  
      | NNode(v, l) -> List.fold_left (fun acc t' -> acc + sum_tree t') v l  
  
      let _ = assert (sum_tree nt = 15)
```

```
[ ]: val sum_tree : int ntree -> int = <fun>
```

```
[ ]: - : unit = ()
```

```
[ ]: val sum_tree : int ntree -> int = <fun>
```

```
[ ]: - : unit = ()
```

Podemos ainda utilizar uma forma mais simples, iterando primeiro a lista de subárvores e depois somando os valores recolhidos.

```
[12]: let rec sum_tree t =
  match t with
  | NEmpty -> 0
  | NNode(v, l) -> List.map sum_tree l |> List.fold_left (+) v

let _ = assert (sum_tree nt = 15)
```

```
[12]: val sum_tree : int ntree -> int = <fun>
```

```
[12]: - : unit = ()
```

### 1.2.3 Mapeamento sobre árvores n-árias — versão 1

`map_ntree f t` aplica `f` a cada valor da árvore preservando a estrutura. Para cada nó, aplica `f` ao valor e mapeia recursivamente sobre a lista de filhos usando uma lambda explícita (`fun t' -> map_ntree f t'`).

```
[14]: let rec map_ntree f nt =
  match nt with
  | NEmpty -> NEmpty
  | NNode (v, l) -> NNode (f v, List.map (fun t' -> map_ntree f t') l)
```

```
[14]: val map_ntree : ('a -> 'b) -> 'a ntree -> 'b ntree = <fun>
```

```
[18]: let _ = assert (map_ntree ((+)1) nt = NNode (2, [NNode (3, [NNode (4, [])]);
↪NNode (5, []); NNode (6, [])]))
```

```
[18]: - : unit = ()
```

### 1.2.4 Mapeamento — versão 2 (estilo point-free)

Versão mais concisa onde a função anónima `fun t' -> map_ntree f t'` passa a ser `map_ntree f` que denota a função que espera o argumento que representa a árvore.

```
[42]: let rec map_ntree f nt =
  match nt with
  | NEmpty -> NEmpty
  | NNode (v, l) -> NNode (f v, List.map (map_ntree f) l)
```

```
[42]: val map_ntree : ('a -> 'b) -> 'a ntree -> 'b ntree = <fun>
```

```
[19]: let _ = assert (map_ntree ((+)1) nt = NNode (2, [NNode (3, [NNode (4, [])]);
↪NNode (5, []); NNode (6, [])]))
```

```
[19]: - : unit = ()
```

### 1.2.5 Fold seguindo a pré-ordem sobre árvores n-árias — versão 1 (função auxiliar recursiva)

`prefix_fold_ntree f acc t` percorre a árvore usando **pré-ordem** (raiz antes das subárvores). Variante com a função auxiliar `fold_aux` que itera manualmente sobre a lista de filhos, sem usar `List.fold_left`. Torna o padrão de acumulação mais visível.

```
[34]: let rec prefix_fold_ntree f acc nt =
      let rec fold_aux acc l =
        match l with
        | [] -> acc
        | nt'::nts -> let acc_nt' = prefix_fold_ntree f acc nt' in fold_aux acc_nt' l
      ↪nts
      in
      match nt with
      | NEmpty -> acc
      | NNode (v, l) -> let acc_v = f acc v in fold_aux acc_v l

      let _ = assert (prefix_fold_ntree (+) 0 nt = 15)
      let _ = assert (prefix_fold_ntree (fun acc x -> acc@[x]) [] nt = [1; 2; 3; 4;
      ↪5])
      let _ = assert (List.rev (prefix_fold_ntree (fun acc x -> x::acc) [] nt) = [1;2;
      ↪3;4;5])
```

```
[34]: val prefix_fold_ntree : ('a -> 'b -> 'a) -> 'a -> 'b ntree -> 'a = <fun>
```

```
[34]: - : unit = ()
```

```
[34]: - : unit = ()
```

```
[34]: - : unit = ()
```

### 1.2.6 Fold em pré-ordem — versão 2 (com `fold_left`)

Esta versão usa `fold_left` para percorrer a lista de subárvores e propaga o resultado com um parametro acumulador. Aplica `f acc v` ao valor da raiz, registra o resultado em `acc_v` e depois percorre a lista das subárvores com `fold_left` passando `acc_v` como valor inicial do acumulador. Não é possível fazer o padrão `map` seguido de `fold` porque o processamento de cada uma das subárvores depende dos valores anteriores.

```
[33]: let rec prefix_fold_ntree f acc nt =
  match nt with
  | NEmpty -> acc
  | NNode (v, l) ->
    let acc_v = f acc v in
    List.fold_left (fun acc nt' -> prefix_fold_ntree f acc nt') acc_v l

let _ = assert (prefix_fold_ntree (+) 0 nt = 15)
let _ = assert (prefix_fold_ntree (fun acc x -> acc@[x]) [] nt = [1; 2; 3; 4; 5])
let _ = assert (List.rev (prefix_fold_ntree (fun acc x -> x::acc) [] nt) = [1; 2; 3; 4; 5])
```

```
[33]: val prefix_fold_ntree : ('a -> 'b -> 'a) -> 'a -> 'b ntree -> 'a = <fun>
```

```
[33]: - : unit = ()
```

```
[33]: - : unit = ()
```

```
[33]: - : unit = ()
```

### 1.2.7 Fold em pré-ordem — versão 2 (point-free)

Versão mais compacta com avaliação parcial da função usada no fold:

```
[32]: let rec prefix_fold_ntree f acc nt =
  match nt with
  | NEmpty -> acc
  | NNode (v, l) -> let acc_v = f acc v in List.fold_left (prefix_fold_ntree f) acc_v l

let _ = assert (prefix_fold_ntree (+) 0 nt = 15)
let _ = assert (prefix_fold_ntree (fun acc x -> acc@[x]) [] nt = [1; 2; 3; 4; 5])
let _ = assert (List.rev (prefix_fold_ntree (fun acc x -> x::acc) [] nt) = [1; 2; 3; 4; 5])
```

```
[32]: val prefix_fold_ntree : ('a -> 'b -> 'a) -> 'a -> 'b ntree -> 'a = <fun>
```

```
[32]: - : unit = ()
```

```
[32]: - : unit = ()
```

```
[32]: - : unit = ()
```

### 1.2.8 Construir uma BST a partir de uma árvore n-ária

Aplica `insert` como função de fold: percorre `nt` em pré-ordem e insere cada valor numa BST inicialmente vazia (`Leaf`). Demonstra que `prefix_fold_ntree` é suficientemente geral para combinar tipos de dados diferentes.

```
[42]: type 'a tree = Empty | Node of 'a * 'a tree * 'a tree

let rec insert_bst x t =
  match t with
  | Empty -> Node(x, Empty, Empty)
  | Node(y, l, r) -> if x <= y then Node(y, insert_bst x l, r) else
    Node(y, l, insert_bst x r)

let _ = prefix_fold_ntree (fun bst x -> insert_bst x bst) Empty nt

let flip f = fun x y -> f y x
let _ = prefix_fold_ntree (flip insert_bst) Empty nt
```

```
[42]: type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

```
[42]: val insert_bst : 'a -> 'a tree -> 'a tree = <fun>
```

```
[42]: - : int tree =
  Node (1, Empty,
    Node (2, Empty, Node (3, Empty, Node (4, Empty, Node (5, Empty, Empty)))))
```

```
[42]: val flip : ('a -> 'b -> 'c) -> 'b -> 'a -> 'c = <fun>
```

```
[42]: - : int tree =
  Node (1, Empty,
    Node (2, Empty, Node (3, Empty, Node (4, Empty, Node (5, Empty, Empty)))))
```